# SIMATS ENGINEERING
## ASSESSMENT TOOL 3

## COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

---

**COURSE OUTCOME COVERED**

**CO5:** *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

*Assessment Tool Weightage: 20%*

---

**Duration : 1 Hour**

**QUESTIONS: 5** **Total Marks:50**

Annexure A

## Reverse Engineering

---

***Code of Conduct:***

*I ATHIN SIVA.S (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

# AI-Based Intelligent Maze Navigation and Optimal Path Planning System

## 1. System Decomposition & Analysis

The AI-Based Intelligent Maze Navigation and Optimal Path Planning System can be decomposed into several functional components. The first component is the User Input module, where the user provides the maze size and required settings. The Maze Generation module then creates a valid maze environment. The generated environment is converted into a searchable grid through Environment Mapping. The AI Pathfinding Engine applies the A* search algorithm to evaluate possible routes. Obstacle Detection identifies blocked cells and supports path updates when obstacles affect navigation. Optimal Path Planning selects the efficient route from the start point to the goal. Finally, the Visualization module displays the maze and selected path, while Performance Analysis measures factors such as path length and navigation time. This decomposition shows how individual components work together to produce intelligent maze navigation.

## 2. Architecture Reconstruction

The reconstructed architecture of the system follows a sequential processing pipeline. The user first provides the maze size and settings. These inputs are passed to the Maze Generation module, which produces the maze environment. The environment is then mapped into a grid so that each cell can be evaluated by the pathfinding algorithm. The AI Pathfinding Engine receives this grid and performs A* search. During the search, the system evaluates nodes using the cost function $f(n) = g(n) + h(n)$. The system expands neighboring nodes and updates their costs. Obstacle Detection identifies blocked cells and prevents the pathfinder from selecting invalid routes. Once the goal is reached, the optimal path is reconstructed and displayed visually. Performance metrics are then analyzed to evaluate the efficiency of the navigation process.

## 3. Algorithm Identification

The primary algorithm identified from the presentation is the A* (A-Star) Search Algorithm. A* is used to find an optimal path through the generated maze. Its evaluation function is $f(n) = g(n) + h(n)$, where $g(n)$ is the actual distance from the starting node to the current node and $h(n)$ is the estimated distance from the current node to the goal. The algorithm starts by placing the start node in the open list. It selects the node having the lowest $f(n)$, examines its neighboring nodes, and updates their costs when a better route is found. The process is repeated until the goal node is reached. The resulting sequence of nodes forms the selected shortest or optimal path. In the presented system, A* provides informed search rather than relying only on slow brute-force exploration.

## 4. Data Flow & Process Mapping

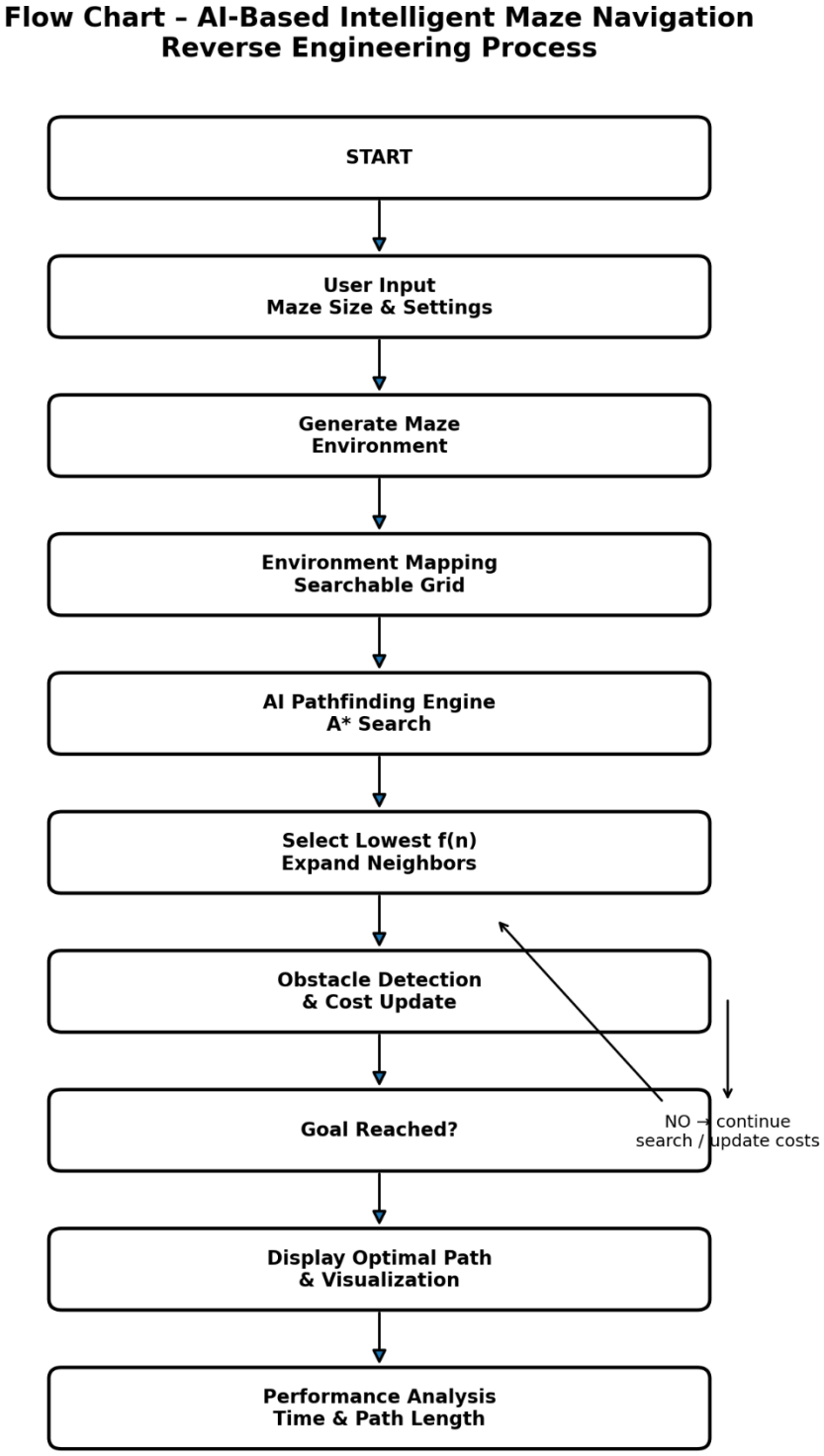
The data flow begins with user input containing the maze size and settings. The Maze Generation module uses these inputs to create a valid maze. The generated maze is represented as a searchable grid through environment mapping. This grid is supplied to the AI Pathfinding Engine. A* evaluates the available cells and calculates the cost of possible movements. During processing, the system checks neighboring cells and detects obstacles or blocked cells. If an obstacle prevents a route, the relevant costs and navigation decisions are updated. The search continues until the goal is found. After the route is finalized, the system displays the maze together with the selected path. Performance Analysis then examines the result using measures such as path length and processing or navigation time.

## 5. Improvement & Redesign

The reverse-engineered system can be enhanced while retaining its existing A* based pathfinding structure. Dynamic obstacle handling can be improved so that the route is updated immediately when the environment changes. Deep Reinforcement Learning can be introduced as a future intelligent decision-making approach. Multi-robot navigation can extend the system so that several autonomous agents can coordinate their routes. IoT integration can connect the navigation system

with external sensors and devices. The system can also be redesigned for 3D maze navigation to represent more complex environments. These improvements can increase adaptability, scalability, and real-time navigation capability while preserving the core objective of efficient path planning.

FLOW CHART – REVERSE ENGINEERING OF AI MAZE NAVIGATION




Flow Chart Explanation

1. START – The system begins the maze navigation process.
2. User Input – Maze size and navigation settings are provided.
3. Generate Maze – A valid maze environment is created.
4. Environment Mapping – The maze is represented as a searchable grid.
5. A* Search – The pathfinding engine evaluates nodes using f(n) = g(n) + h(n).

6. Expand and Update – Neighboring nodes are expanded and their costs are updated.

7. Obstacle Detection – Blocked cells are identified and unsuitable routes are avoided.

8. Goal Decision – If the goal is not reached, the search continues; otherwise the route is finalized.

9. Visualization – The optimal path is displayed through the maze.

10. Performance Analysis – Path length and navigation/processing performance are analyzed.

Source Basis: The answers and flow are based on the supplied AI-Based Intelligent Maze Navigation and Optimal Path Planning System presentation, including its system architecture, modules, A* algorithm, applications, advantages, and future scope.

**RUBRICS FOR CALCULATION**

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Industry Problem | 20% | Comprehensive understanding of real-world problem and constraints | Good understanding with minor gaps in requirements | Basic understanding; some requirements unclear | Poor understanding; misinterprets problem context |
| Application of Engineering Concepts | 25% | Applies advanced and relevant concepts effectively to solve the problem | Applies appropriate concepts with minor errors | Limited application of concepts | Incorrect or no application of concepts |
| Solution Design | 25% | Innovative, practical, and feasible solution aligned with industry standards | Logical and feasible solution with minor gaps | Basic solution with limited feasibility | Unclear or impractical solution |
| Use of Tools | 15% | Effective use of modern tools, technologies, and real data | Appropriate use of tools with correct implementation | Limited or inconsistent use of tools | Minimal or incorrect use of tools |
| Analysis | 10% | Thorough analysis with validated results and performance evaluation | Good analysis with reasonable validation | Basic analysis with limited validation | Poor or no analysis |
| Professionalism | 5% | Highly professional, well-documented, and clearly presented work | Good documentation and presentation | Basic documentation with some gaps | Poor documentation and presentation |